

# Guia de Referência Rápida de Git

Uso profissional no dia a dia

*Consulta rápida para o fluxo de trabalho diário*

---

## Sumário

## 1. Setup Inicial

Configuração feita uma vez por máquina. Use --global para valer em todos os repositórios do usuário; omita para configurar apenas o repositório atual.

### Usuário e e-mail

```
git config --global user.name "Seu Nome"
git config --global user.email "voce@empresa.com"

# Conferir a configuração atual
git config --list
git config user.email # valor efetivo neste repositório
```

### Editor e qualidade de vida

```
# Define o editor padrão (ex.: VS Code aguardando o fechamento)
git config --global core.editor "code --wait"

# Cores na saída e branch padrão como main
git config --global color.ui auto
git config --global init.defaultBranch main
```

### Chaves SSH

Gere uma chave, registre no agente e adicione a pública no GitHub/GitLab para autenticar sem senha.

```
# 1) Gerar a chave (ed25519 é o padrão recomendado)
ssh-keygen -t ed25519 -C "voce@empresa.com"

# 2) Iniciar o ssh-agent e adicionar a chave
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# 3) Copiar a chave PÚBLICA e colar nas configs do servidor
cat ~/.ssh/id_ed25519.pub

# 4) Testar a conexão
ssh -T git@github.com
```

| Flag              | O que faz                                                               |
|-------------------|-------------------------------------------------------------------------|
| <b>-t ed25519</b> | Define o tipo/ algoritmo da chave (ed25519: moderno e seguro).          |
| <b>-C "..."</b>   | Adiciona um comentário, normalmente o e-mail, para identificar a chave. |
| <b>-T</b>         | Testa a autenticação sem abrir um shell remoto.                         |

## 2. Fluxo de Trabalho Diário

A sequência lógica do dia a dia: trazer o que mudou, alterar, registrar e enviar.

### Clonar um repositório (uma vez)

```
git clone git@github.com:org/repositorio.git

# Clonar para uma pasta com outro nome
git clone git@github.com:org/repositorio.git minha-pasta
```

## O ciclo do dia a dia

```
# 1) Atualizar sua branch com o remoto antes de começar
git pull

# 2) Ver o que mudou
git status      # arquivos modificados/não rastreados
git diff        # diferenças ainda não adicionadas ao stage

# 3) Adicionar mudanças ao stage
git add caminho/arquivo.js # arquivo específico
git add .          # tudo a partir da pasta atual

# 4) Commitar
git commit -m "feat: adiciona validação de login"

# 5) Enviar para o remoto
git push
```

| Flag                | O que faz                                                                    |
|---------------------|------------------------------------------------------------------------------|
| <b>-m</b>           | Define a mensagem do commit direto na linha (sem abrir o editor).            |
| <b>git add .</b>    | Adiciona todas as mudanças da pasta atual e subpastas ao stage.              |
| <b>-u (no push)</b> | Vincula a branch local ao remoto na 1ª vez: git push -u origin minha-branch. |

## 3. Gestão de Branches

Trabalhe sempre em uma branch separada da principal (main). Crie, alterne, liste e remova com segurança.

### Criar e alternar

```
# Criar e já mudar para a nova branch (forma moderna)
git switch -c feature/login

# Equivalente clássico
git checkout -b feature/login

# Alternar para uma branch existente
git switch main
```

### Listar

```
git branch      # branches locais (a atual vem com *)
git branch -a   # locais + remotas
git branch -vv  # mostra o último commit e a branch remota associada
```

### Enviar uma branch nova e atualizar

```
# Primeira vez: cria a branch no remoto e vincula
git push -u origin feature/login

# Trazer atualizações de outras branches
git fetch      # baixa referências sem mexer no seu trabalho
```

### Mesclar (merge) com segurança

```
# 1) Garanta que a branch de destino está atualizada
```

```
git switch main
git pull
```

```
# 2) Traga as mudanças da feature para a main
git merge feature/login
```

```
# Forçar um commit de merge mesmo quando seria fast-forward
git merge --no-ff feature/login
```

## Deletar

```
# Deletar branch local já mesclada (seguro)
git branch -d feature/login
```

```
# Forçar deleção mesmo sem merge (cuidado: perde commits)
git branch -D feature/login
```

```
# Deletar a branch no remoto
git push origin --delete feature/login
```

| Flag           | O que faz                                                             |
|----------------|-----------------------------------------------------------------------|
| <b>-c</b>      | Cria a branch e já alterna para ela (git switch -c).                  |
| <b>-b</b>      | Equivalente ao -c no comando clássico git checkout -b.                |
| <b>-d</b>      | Deleta a branch apenas se já foi mesclada (proteção contra perda).    |
| <b>-D</b>      | Força a deleção mesmo sem merge — use com cautela.                    |
| <b>--no-ff</b> | Cria um commit de merge explícito, preservando o histórico da branch. |
| <b>-a</b>      | Lista também as branches remotas.                                     |

## 4. Resolução de Conflitos

Um conflito surge quando duas alterações tocam as mesmas linhas. O git pausa o merge/pull e marca os trechos para você decidir.

### Passo a passo

1. Rode o git pull ou git merge. Se houver conflito, o git avisa: "Automatic merge failed; fix conflicts and then commit".
2. Identifique os arquivos em conflito com git status (aparecem como "both modified").
3. Abra cada arquivo e procure os marcadores de conflito (veja abaixo).
4. Edite manualmente: escolha o seu código, o do outro, ou combine os dois — e remova os marcadores.
5. Marque como resolvido com git add no arquivo.
6. Finalize: git commit (no merge) ou git rebase --continue / git merge --continue conforme o caso.

### Como os marcadores aparecem

```
<<<<<< HEAD
    seu código (a versão da sua branch atual)
=====
    código que está vindo (da outra branch)
>>>>>> feature/login
```

Apague as linhas <<<<<<, ===== e >>>>>> deixando apenas o conteúdo final desejado.

## Comandos úteis durante o conflito

```
git status      # lista os arquivos em conflito
git diff        # mostra os trechos conflitantes
git add arquivo.js  # marca o arquivo como resolvido
git commit      # conclui o merge

# Desistir e voltar ao estado anterior ao merge
git merge --abort

# Aceitar inteiramente um dos lados em um arquivo
git checkout --ours arquivo.js  # mantém a sua versão
git checkout --theirs arquivo.js # mantém a versão que veio
```

| Flag            | O que faz                                                       |
|-----------------|-----------------------------------------------------------------|
| <b>--abort</b>  | Cancela o merge/rebase em andamento e volta ao estado anterior. |
| <b>--ours</b>   | Resolve o arquivo mantendo a versão da sua branch atual.        |
| <b>--theirs</b> | Resolve o arquivo mantendo a versão que está sendo integrada.   |

## 5. Comandos "Salva-Vidas"

Para desfazer erros comuns sem pânico.

### Descartar mudanças locais em um arquivo

Reverte o arquivo para o último commit. Atenção: o trabalho não commitado é perdido.

```
git restore caminho/arquivo.js # forma moderna

# Descartar TODAS as mudanças não commitadas (cuidado)
git restore .

# Equivalente clássico
git checkout -- caminho/arquivo.js
```

### Remover um arquivo do stage

Tira o arquivo do stage mantendo suas edições no diretório de trabalho.

```
git restore --staged caminho/arquivo.js

# Equivalente clássico
git reset HEAD caminho/arquivo.js
```

### Alterar a mensagem do último commit

Só faça isso se o commit ainda NÃO foi enviado (push). Reescrever histórico já compartilhado quebra o de outros.

```
# Trocar a mensagem do último commit
git commit --amend -m "fix: mensagem corrigida"

# Adicionar arquivos esquecidos ao último commit (sem mudar a mensagem)
git add arquivo-esquecido.js
git commit --amend --no-edit
```

## Usar git stash para trocar de branch

Guarda temporariamente o trabalho inacabado, deixando a árvore limpa para você trocar de branch. Depois você recupera.

```
# Guardar o trabalho atual (inclui stage e modificações)
git stash
```

```
# Guardar com um rótulo descritivo
git stash push -m "wip: formulário de login"
```

```
# Trocar de branch, fazer o que precisa, e voltar...
git switch outra-branch
```

```
# Ver a pilha de stashes
git stash list
```

```
# Recuperar o último stash e removê-lo da pilha
git stash pop
```

```
# Aplicar sem remover da pilha
git stash apply
```

| Flag                 | O que faz                                                               |
|----------------------|-------------------------------------------------------------------------|
| <b>--amend</b>       | Refaz o último commit (mensagem e/ou arquivos) em vez de criar um novo. |
| <b>--no-edit</b>     | Mantém a mensagem atual ao usar --amend (não abre o editor).            |
| <b>--staged</b>      | Em git restore, remove o arquivo do stage preservando as edições.       |
| <b>-m (no stash)</b> | Adiciona uma descrição ao stash para identificá-lo na lista.            |
| <b>pop / apply</b>   | pop recupera e remove o stash; apply recupera e mantém na pilha.        |

## 6. Boas Práticas de Commit

Mensagens claras ajudam o time a entender o histórico e automatizar releases. O padrão Conventional Commits é o mais usado.

### Formato

```
<tipo>(<escopo opcional>): <descrição curta no imperativo>
```

```
[corpo opcional explicando o porquê]
```

```
[rodapé opcional: BREAKING CHANGE, refs de issue]
```

### Tipos mais comuns

- feat: nova funcionalidade para o usuário.
- fix: correção de bug.
- docs: apenas documentação.
- style: formatação que não altera o comportamento (espaços, ponto e vírgula).
- refactor: mudança de código que não corrige bug nem adiciona feature.
- test: adiciona ou ajusta testes.

- chore: tarefas de manutenção, build, dependências.
- perf: melhoria de performance.

## Exemplos

```
feat(auth): adiciona login via SSO  
fix(api): corrige timeout em requisições longas  
docs(readme): atualiza instruções de setup  
refactor(checkout): extrai cálculo de frete para serviço
```

```
# Mudança incompatível (quebra contrato)  
feat(api): altera formato de resposta do endpoint /users
```

```
BREAKING CHANGE: o campo 'name' foi dividido em 'firstName' e 'lastName'.
```

## Regras práticas

- Use o imperativo: "adiciona", "corrige" — não "adicionado" ou "adicionando".
- Primeira linha curta (até ~50 caracteres) e sem ponto final.
- Deixe uma linha em branco antes do corpo; explique o porquê, não o como.
- Um commit por mudança lógica — evite misturar refatoração com feature.
- Referencie issues no rodapé quando aplicável (ex.: Closes #123).